Adding SEMI JOIN and ANTI JOIN to SQL in PostgreSQL

Final Semester Project — Detailed Plan for a Team of Four

Team of four:
Member 1 (Parser) Member 2 (Planner)
Member 3 (Rewriter/Deparse/Executor) Member 4 (QA/Docs/PM)

October 26, 2026

Abstract

SQL has no syntax for the semijoin and the anti-semijoin, yet PostgreSQL's planner and executor already implement both operators (they are produced today when `EXISTS`, `NOT EXISTS`, and `IN` sublinks are pulled up into joins). This project adds *user-visible syntax* `A SEMI JOIN B ON ...` and `A ANTI JOIN B ON ...`, wires it through the parser, rewriter, deparser (views/`pg_dump`) and planner, reuses the existing executor nodes, and validates the result with regression tests, differential testing against the `EXISTS` formulation, and benchmarks. This document lists every task, who does it, which files it touches, *why* it is needed, and how we know it is finished. It is based on a scan of the PostgreSQL source tree (version string `20devel`, commit `022a80f`); line numbers quoted below refer to that commit and will drift on other branches.

Contents

1 Goal, Scope and Definitions **3**
1.1 Problem statement . . . 3
1.2 In scope (Minimum Viable Product) . . . 3
1.3 Stretch goals (only after MVP is merged on our branch) . . . 3
1.4 Non-goals . . . 3
1.5 Semantic rules we commit to . . . 3

2 Findings from the Repository Scan **4**
2.1 What already exists vs. what is missing . . . 4
2.2 Pipeline view and where we change code . . . 5

3 Proposed Syntax and Grammar Sketch **5**

4 Key Design Decisions (to be closed in Phase 1) **6**

5 Work Breakdown Structure **6**

6 Team Organisation **16**

7 Timeline (12 Weeks) **16**

8 Risk Register **17**

9 Definition of Done **17**

A File Map: Where to Look in the Repository **18**

B Developer Cheat-Sheet **19**

C Seed Test Queries **19**

1 Goal, Scope and Definitions

1.1 Problem statement

For relations R and S and a predicate θ:

$$R \ltimes_\theta S = \{\, r \in R \mid \exists s \in S : \theta(r, s) \,\} \qquad \text{(semijoin)}$$
$$R \triangleright_\theta S = \{\, r \in R \mid \nexists s \in S : \theta(r, s) \,\} \qquad \text{(anti-semijoin)}$$

Both return only columns of R, preserve duplicates of R, and never multiply an R-row because S has several matches. Today users must write `WHERE EXISTS (...)` / `WHERE NOT EXISTS (...)` and hope the planner converts it. Outer joins, by contrast, have first-class syntax. We add the missing syntax.

1.2 In scope (Minimum Viable Product)

- `table_ref SEMI JOIN table_ref {ON cond | USING (cols)}`
- `table_ref ANTI JOIN table_ref {ON cond | USING (cols)}`
- Right-hand-side columns are *not* visible after the join (neither in `SELECT`, `WHERE`, nor `SELECT *`).
- Correct behaviour in views, rules, `pg_dump`/restore, `EXPLAIN`, prepared statements, `UPDATE ...FROM`, `DELETE ...USING`.
- Plans identical (or provably as good) as for the equivalent `EXISTS`/`NOT EXISTS` query.
- Regression tests, documentation, benchmark report.

1.3 Stretch goals (only after MVP is merged on our branch)

- `NATURAL SEMI JOIN`/`NATURAL ANTI JOIN`.
- `LEFT SEMI JOIN` / `LEFT ANTI JOIN` spellings (Spark/Hive style) as synonyms.
- `LATERAL` right-hand side of a semi/anti join.
- `RIGHT SEMI JOIN` / `RIGHT ANTI JOIN` (the executor already supports the right-hand variants, `JOIN_RIGHT_SEMI` and `JOIN_RIGHT_ANTI`).
- psql tab-completion; aliasing a parenthesised semi-join result.

1.4 Non-goals

New executor nodes; changing how `EXISTS` is planned; SQL-standard conformance claims (SEMI/ANTI JOIN are *not* in the SQL standard, so this is a PostgreSQL extension); upstream acceptance on `pgsql-hackers` (optional bonus, not a grading target).

1.5 Semantic rules we commit to

1. **Output columns:** only the left input's columns.
2. **Duplicates:** left rows are returned once per left row, regardless of the number of right matches.

3. **ANTI and NULLs:** `ANTI JOIN` is the analogue of `NOT EXISTS`, *not* of `NOT IN`. A right row with NULL in the join column simply does not match; it does not suppress left rows.
4. **ON placement:** for `ANTI JOIN`, a conjunct of `ON` that references only left columns may not be pushed below the join (it would change results); the same restriction already exists for `LEFT JOIN`. For `SEMI JOIN` it *is* equivalent to a filter.
5. **Missing ON/USING:** syntax error (inherited from the `join_qual` rule).

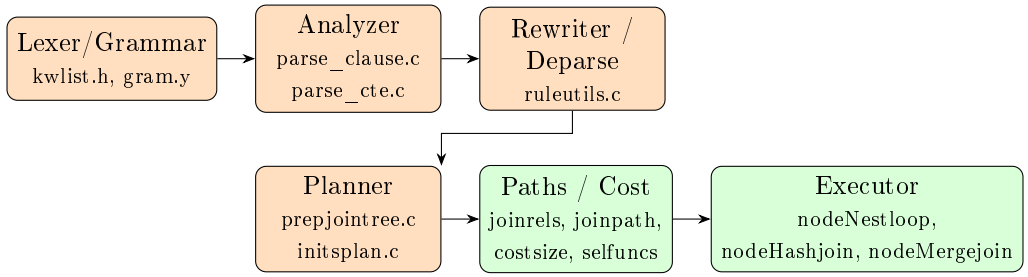
2 Findings from the Repository Scan

2.1 What already exists vs. what is missing

Component	Already in the tree	Gap for this project
Join type enum	`src/include/nodes/nodes.h`: `JOIN_SEMI`, `JOIN_ANTI`, `JOIN_RIGHT_SEMI`, `JOIN_RIGHT_ANTI` already defined. Comment there says only INNER/LEFT/FULL/RIGHT "can appear in parser output".	Comment becomes false; update it, and re-audit code that relied on that promise.
Grammar	`src/backend/parser/gram.y`: `join_type` has only `FULL`, `LEFT`, `RIGHT`, `INNER`. `joined_table` rules reuse `join_type` for `NATURAL`.	New productions; new keywords.
Keywords	`SEMI` and `ANTI` are *not* in `src/include/parser/kwlist.h` (verified by grep; no conflicts in `gram.y`).	Add both; choose category (Decision D1).
Semantic analysis	`parse_clause.c`, `transformFromClauseItem`: `switch (j->jointype)` with `default: elog(ERROR, "unrecognized join type")`; `lateral_ok` only for INNER/LEFT; `buildMergedJoinVar` also switches.	New cases; hide right-side namespace items; left-only output columns.
Recursive CTE check	`parse_cte.c` `checkWellFormedRecursionWalker`: switch with `default: elog(ERROR)`.	Add SEMI/ANTI cases.
Deparse	`utils/adt/ruleutils.c` `get_from_clause_item`: prints `JOIN`, `LEFT JOIN`, `FULL JOIN`, `RIGHT JOIN`; `default: elog(ERROR)`.	Print `SEMI JOIN`/`ANTI JOIN`, otherwise views and `pg_dump` break.
Sublink pull-up (template)	`optimizer/plan/subselect.c`: `convert_ANY_sublink_to_join`, `convert_EXISTS_sublink_to_join` build a `JoinExpr` with `JOIN_SEMI`/`JOIN_ANTI`; semijoin gets `rtindex = 0`; antijoin gets a column-less join RTE via `add_antijoin_rte()`.	This is exactly the shape the planner already accepts; our parser output should converge to it (Decision D2).

Component	Already in the tree	Gap for this project
Pull-up pass	`optimizer/prep/prepjointree.c` `pull_up_sublinks_jointree_recurse`: comment "we don't expect to see any pre-existing `JOIN_SEMI`, ..." and a `default: elog(ERROR)` (around lines 800–830).	Must accept parser-made semi/anti joins. Highest-risk change.
Join-order logic	`initsplan.c` (`deconstruct_recurse`, `make_outerjoininfo`, cases at `JOIN_ANTI`/`JOIN_SEMI`), `joinrels.c` (legality, `populate_joinrel_with_paths`), `joinpath.c`, `costsize.c`, `utils/adt/selfuncs.c`.	Verify they are source-agnostic; fix hidden assumptions.
Executor	`executor/nodeNestloop.c`, `nodeHashjoin.c`, `nodeMergejoin.c` implement SEMI/ANTI (and RIGHT variants). `commands/explain.c` labels them.	No change expected; must be verified by tests.
Documentation	`doc/src/sgml/ref/select.sgml` synopsis (lines 62–63) and `join_type` section (around line 591).	Document new syntax.
Tests	`src/test/regress/sql/join.sql`, `subselect.sql`, `rules.sql`; schedule in `src/test/regress/parallel\_schedule`.	New test file + expected output.

2.2 Pipeline view and where we change code



Orange = code changes required. Green = already supports SEMI/ANTI; verify only.

3 Proposed Syntax and Grammar Sketch

Sketch only; it has **not** been compiled. Real conflicts will be discovered in task T2.2.

```
/* src/backend/parser/gram.y -- existing rule, extended */
join_type:      FULL opt_outer      { $$ = JOIN_FULL; }
                | LEFT opt_outer        { $$ = JOIN_LEFT; }
                | RIGHT opt_outer       { $$ = JOIN_RIGHT; }
                | INNER_P               { $$ = JOIN_INNER; }
                | SEMI                  { $$ = JOIN_SEMI; }   /* new */
                | ANTI                  { $$ = JOIN_ANTI; }   /* new */
        ;
```

```
/* src/include/parser/kwlist.h (must stay alphabetical) */
PG_KEYWORD("anti", ANTI, UNRESERVED_KEYWORD, BARE_LABEL)
PG_KEYWORD("semi", SEMI, UNRESERVED_KEYWORD, BARE_LABEL)
```

Known grammar hazard. `alias_clause` allows an alias without `AS` (`ColId`), and `ColId` includes unreserved keywords. So in `FROM a semi JOIN b` the parser cannot tell whether `semi` is an alias for `a` or the start of `SEMI JOIN`. That is why `LEFT`, `FULL`, `NATURAL` are in the `TYPE_FUNC_NAME_KEYWORD` category and appear in the `%left JOIN CROSS LEFT FULL RIGHT INNER_P NATURAL` precedence line of `gram.y`. We will most likely have to follow the same route (Decision D1).

4 Key Design Decisions (to be closed in Phase 1)

ID	Question	Options and recommended starting point
D1	Keyword category for `SEMI`, `ANTI`	(a) `UNRESERVED`: least breaking for existing schemas, but likely creates shift/reduce conflicts with alias-without-`AS`; may be fixable with precedence declarations. (b) `TYPE_FUNC_NAME_KEYWORD`: same as `LEFT`/`NATURAL`; conflict-free, but tables/columns named `semi`/`anti` then need quoting. *Start with (a); fall back to (b) if bison reports conflicts we cannot resolve cleanly. Run `check_keywords.pl` either way.*
D2	Query-tree shape of the join	(a) Mimic pull-up output: `JoinExpr.rtindex = 0` for SEMI, a column-less join RTE for ANTI. Fewest planner surprises, but a parenthesised, aliased semi-join cannot expose columns. (b) Build a normal `RTE_JOIN` with left-only columns: supports aliases and `SELECT *` through an alias, but the planner has never seen this shape for SEMI. *Prototype (a) first (MVP), keep (b) as stretch.* Decision is made by T1.3 after a one-day spike.
D3	`USING` semantics	Equality on named columns, right-side copy not exposed; merged column comes from the left. Confirm `buildMergedJoinVar` handling.
D4	`LATERAL` on the right	MVP: reject with a clear error (the planner support for semijoin RHS that references the LHS other than through `ON` is unverified). Stretch: allow.
D5	Recursive CTE reference on the right	Mirror `LEFT JOIN` rules: reference allowed on the left, rejected on the right (ANTI is non-monotonic; SEMI kept symmetric for simplicity).
D6	Catalog impact	As long as we add no field to `JoinExpr`/`RangeTblEntry`, node serialisation (`gen_node_support.pl`) and stored rules need no `catversion` bump. If D2(b) forces a new field, bump `src/include/catalog/catversion.h`.

5 Work Breakdown Structure

Phases: P0 setup → P1 design → P2 parser → P3 rewriter/deparse → P4 planner → P5 test/benchmark → P6 docs/polish → P7 stretch → P8 final deliverables. P5 starts in week 3 and runs continuously (tests are written *with* features, not after).

Phase 0 — Setup and shared understanding (Week 1)

T0.1	**Build PostgreSQL from source with debug flags**
Owner / depends on	All / –
Files	`configure.ac`, `meson.build`
What	Install bison, flex, perl, readline/zlib headers, gdb. Configure with `-enable-debug -enable-cassert -enable-tap-tests` and `CFLAGS="-O0 -g"` into a private prefix; `make -j`, `make check`. Use a fixed data directory per member.
Why	Assertions (`-enable-cassert`) are what will catch planner invariants we violate. Without a working build nobody can test anything.
Done when	Every member can run `make check` green and attach gdb to a backend.

T0.2	**Pin the source version and set up Git workflow**
Owner / depends on	Member 4 / –
Files	repository root
What	Fork the official repo, pin a commit (or latest stable branch), create `main`, `feature/parser`, `feature/planner`, `feature/deparse`, `feature/tests-docs`. Enforce PRs with one cross-review. Agree on commit-message style.
Why	`gram.y` and `parse_clause.c` are hot spots; four people editing them without discipline guarantees merge pain. Pinning avoids chasing upstream while we work.
Done when	Branches exist, CI or a local script runs `make check` on each PR.

T0.3	**Baseline measurements**
Owner / depends on	Member 4 / T0.1
Files	`src/test/regress/sql/subselect.sql`, `join.sql`
What	Run the full regression suite and save the output. For a toy schema record `EXPLAIN (COSTS OFF)` for `EXISTS`, `NOT EXISTS`, `IN`, `NOT IN`, and `LEFT JOIN ... IS NULL`. Note which show `Semi Join`/`Anti Join` and which do not (e.g. `NOT IN` is not converted).
Why	Defines the "golden" plans our new syntax must match, and proves failures later are ours, not pre-existing.
Done when	Baseline plans and test-suite results stored in the repo under `docs/baseline/`.

T0.4	**Subsystem reading notes (one page each)**
Owner / depends on	M1 parser, M2 planner, M3 rewriter/executor, M4 tests/docs / T0.1
Files	M1: `gram.y`, `parse\_clause.c`, `parse\_relation.c`. M2: `prepjointree.c`, `initsplan.c`, `subselect.c`, `optimizer/README`. M3: `ruleutils.c`, `rewriteHandler.c`, `nodeHashjoin.c`. M4: `regress/` harness, `select.sgml`.
What	Each member traces one existing `LEFT JOIN` query through their subsystem with `debug_print_parse` / `debug_print_rewritten` / `debug_print_plan` and writes a one-page note, including a call chain.
Why	We are modifying a 20-year-old code base; the fastest way to avoid wrong fixes is to know how the closest feature (outer join) is implemented.
Done when	Four notes committed; 30-minute walkthrough held.

Phase 1 — Language design (Week 2)

T1.1	**Write the syntax and semantics specification**
Owner / depends on	Member 1 (+ M4 reviews) / T0.4
Files	new `docs/spec.md`
What	Formal grammar, name-visibility rules, NULL behaviour, `ON`/`USING`, chaining (`A SEMI JOIN B ... ANTI JOIN C ...`), interaction with `LEFT`/`INNER` joins, DML (`UPDATE FROM`, `DELETE USING`, `MERGE USING`), and the error messages we expect.
Why	A written contract lets parser, planner and test people work in parallel and gives M4 an oracle for the expected outputs.
Done when	Spec approved by all four members.

T1.2	**Resolve D1 (keyword category) with a bison experiment**
Owner / depends on	Member 1 / T1.1
Files	`gram.y`, `kwlist.h`, `src/backend/parser/check\_keywords.pl`
What	Add `SEMI`/`ANTI` as unreserved on a scratch branch, run bison, read the conflict report; try precedence declarations; if unfixable move to `TYPE_FUNC_NAME_KEYWORD`. Record the outcome and the compatibility impact.
Why	This is the single most likely source of grammar trouble, and the answer changes whether existing user schemas keep working.
Done when	D1 closed with evidence (bison output attached to the decision).

T1.3	**Resolve D2 (query-tree shape) with a spike**
Owner / depends on	Members 1+2 / T1.1
Files	`parse\_clause.c`, `subselect.c`, `prepjointree.c`, `initsplan.c`
What	Hand-construct (in a debugger or by a throw-away hack in the parser) a `JoinExpr(JOIN_SEMI)` with `rtindex=0` and see whether the planner survives; repeat for ANTI with a join RTE. Record every assertion/`elog` hit.
Why	Planner failures are expensive to discover late. One day of spiking de-risks Phases 3 and 4.
Done when	D2 closed; list of known planner sites needing changes (feeds T4.1).

T1.4	**Define the test oracle**
Owner / depends on	Member 4 / T1.1
Files	`src/test/regress/`
What	Formalise the rule: the result of `A SEMI JOIN B ON p` must equal that of `SELECT * FROM A WHERE EXISTS (SELECT 1 FROM B WHERE p)`; likewise ANTI against `NOT EXISTS`. Compare as multisets, ordered by all columns.
Why	An executable definition of "correct" makes differential testing (T5.2) possible without hand-written expected outputs for everything.
Done when	Oracle documented with 5 worked examples.

Phase 2 — Parser and semantic analysis (Weeks 3–4)

T2.1	Register the keywords
Owner / depends on	Member 1 / T1.2
Files	`src/include/parser/kwlist.h`
What	Add `anti` and `semi` in alphabetical order with the chosen category; rebuild to regenerate the perfect-hash keyword tables.
Why	The scanner cannot emit tokens it does not know; `gen_keywordlist.pl` assumes sorted input.
Done when	`SELECT 1 AS semi` and quoted identifiers still work; `check_keywords.pl` passes.

T2.2	Extend the grammar
Owner / depends on	Member 1 / T2.1
Files	`src/backend/parser/gram.y`: `%token` list, `join_type`, `unreserved_keyword`/`bare_label_keyword` lists, precedence line (`%left JOIN CROSS LEFT ...`)
What	Add the productions from Section 3; keep `opt_outer` untouched; make sure `NATURAL join_type JOIN` either supports or cleanly rejects SEMI/ANTI. Drive bison to zero conflicts (the file declares an expected conflict count of zero).
Why	Without this nothing parses. A conflict hidden by a quick precedence hack can silently change how existing queries parse, so conflicts must be understood, not suppressed.
Done when	`make` succeeds with no new bison warnings; existing regression suite unchanged; `EXPLAIN` not yet required.

T2.3	ECPG preprocessor
Owner / depends on	Member 4 / T2.2
Files	`src/interfaces/ecpg/preproc/` (`parse.pl`, `ecpg.trailer`, `ecpg.addons`)
What	The ECPG grammar is generated from `gram.y` by `parse.pl`. Rebuild `ecpg` and run `make -C src/interfaces/ecpg check`. Fix generation errors if the new rules need an ECPG-specific rule.
Why	It is the one consumer of `gram.y` that is easy to forget; a green backend with a broken ECPG fails `make check-world`.
Done when	ECPG builds and its tests pass.

T2.4	**Analyzer: transform SEMI/ANTI joins**
Owner / depends on	Member 1 / T2.2, T1.3
Files	`src/backend/parser/parse_clause.c`: `transformFromClauseItem`, `buildMergedJoinVar`, `markRelsAsNulledBy` switch (around lines 1430–1450)
What	(i) Add SEMI/ANTI cases to both switches; no "nulled-by" marking is needed for either side because the right side is not referenceable afterwards. (ii) After processing, *do not* export the right-hand namespace items to the enclosing scope, so that `b.x` errors and `SELECT *` expands to left columns only. (iii) Keep `lateral_ok` false for SEMI/ANTI (D4). (iv) Set `rtindex`/RTE according to D2. (v) Make `USING` behave per D3.
Why	This is the core semantic change. The "right side not visible" rule is what distinguishes a semijoin from an inner join; if it is wrong, users can read columns the executor never produces.
Done when	Queries in Appendix C parse and produce the expected tree under `debug_print_parse`; referencing a right-side column gives `missing FROM-clause entry` or an equivalent error.

T2.5	**Join RTE for left-only columns (only if D2 = b)**
Owner / depends on	Member 1 / T2.4
Files	`src/backend/parser/parse\_relation.c` (`addRangeTableEntryForJoin`), `src/include/nodes/parsenodes.h`
What	Create `RTE_JOIN` entries whose `joinaliasvars` cover only left columns (`joinrightcols = NIL`).
Why	Required to support `(A SEMI JOIN B ON ...) AS x`.
Done when	Aliased semi-join works end to end. Skipped entirely if D2 = a.

T2.6	**Recursive-CTE well-formedness check**
Owner / depends on	Member 1 / T2.2, D5
Files	`src/backend/parser/parse\_cte.c` `checkWellFormedRecursionWalker` (around line 1148)
What	Add SEMI/ANTI cases following the `LEFT JOIN` model: recursion allowed on the left, forbidden on the right.
Why	Otherwise `WITH RECURSIVE` + SEMI JOIN hits `elog(ERROR, "unrecognized join type")`, an internal error instead of a proper message, and ANTI on the recursive side would give non-monotone results.
Done when	`WITH RECURSIVE` tests for both positions pass with the intended behaviour.

T2.7	**Update the comment on `JoinType`**
Owner / depends on	Member 1 / T2.4
Files	`src/include/nodes/nodes.h`
What	Replace "only these codes can appear in parser output" with an accurate statement.
Why	Stale invariants in comments mislead future maintainers and reviewers.
Done when	Comment updated in the same PR as T2.4.

Phase 3 — Rewriter, deparse, dump/restore (Week 5)

T3.1	Deparse SEMI/ANTI joins
Owner / depends on	Member 3 / T2.4
Files	`src/backend/utils/adt/ruleutils.c`: `get_from_clause_item` (lines 12777–12810 and the `ON TRUE` fallback at line 12850)
What	Add `" SEMI JOIN "` and `" ANTI JOIN "` to the switch. Check column naming helpers (`set_join_column_names`, `set_rtable_names`) cope with a join RTE that has fewer columns, or no RTE at all (D2).
Why	Views and rules are stored as parse trees and printed through this function; `pg_dump` uses `pg_get_viewdef`. Without it, any view using the new syntax dumps as an error.
Done when	`pg_get_viewdef` output re-parses to an identical tree.

T3.2	Rewriter and view expansion
Owner / depends on	Member 3 / T2.4
Files	`src/backend/rewrite/rewriteHandler.c`, `rewriteManip.c` (`OffsetVarNodes`, `ChangeVarNodes`, `IncrementVarSublevelsUp` treat `JoinExpr.rtindex`)
What	Test a view containing SEMI/ANTI used (a) plain, (b) on the nullable side of a `LEFT JOIN`, (c) in a subquery, (d) under `UPDATE`/`DELETE`. Verify `rtindex` shifting is correct when `rtindex = 0`.
Why	View inlining renumbers range-table indexes; a semi-join with `rtindex = 0` is a shape those routines rarely see outside the planner.
Done when	All four view scenarios return oracle-equal results.

T3.3	Dump/restore round trip
Owner / depends on	Member 3 / T3.1
Files	`src/bin/pg\_dump`, `src/bin/pg\_upgrade`
What	Create views using SEMI/ANTI in the regression database (they are then picked up by the pg_upgrade test during `make check-world`); diff `pg_dump` output after restore.
Why	Proves persistence of the feature, not only in-memory behaviour.
Done when	Dump → restore → dump is a fixed point.

T3.4	Query jumbling and `pg_stat_statements`
Owner / depends on	Member 3 / T2.4
Files	`src/include/nodes/primnodes.h` (`JoinExpr` attributes), `contrib/pg\_stat\_statements`
What	Confirm `jointype` participates in the query ID (it is not marked `query_jumble_ignore`) so SEMI and INNER versions of a query are separate entries.
Why	Wrong merging would corrupt monitoring statistics.
Done when	Two statements differing only in SEMI vs INNER produce different `queryid`.

Phase 4 — Planner integration (Weeks 5–7; critical path)

T4.1	**Audit every consumer of `JOIN_SEMI`/`JOIN_ANTI`**
Owner / depends on	Member 2 / T1.3
Files	Files referencing the symbols (from grep): `optimizer/path/\{allpaths,costsize,indxpath,joinpath,joinrels\}.c`, `optimizer/plan/\{analyzejoins,initsplan,planner,subselect\}.c`, `optimizer/prep/prepjointree.c`, `optimizer/util/\{clauses,relnode\}.c`, `partitioning/partbounds.c`, `utils/adt/\{selfuncs,network\_selfuncs\}.c`, `commands/explain.c`, executor `node\{Nestloop,Hashjoin,Mergejoin\}.c`
What	Build a table: file, function, assumption ("semijoins come only from sublinks"), risk, verdict. Specifically look for code that assumes a semijoin's `rtindex` is 0, that its RHS was a pulled-up subquery, or that `ON` quals reference only certain rels.
Why	We are the first source of semi/anti joins that is not the pull-up code. Hidden assumptions will show up as wrong plans or assertion failures, not as compile errors.
Done when	Audit table complete and reviewed by Member 3.

T4.2	**Accept parser-made semi/anti joins in `prepjointree.c`**
Owner / depends on	Member 2 / T4.1
Files	`optimizer/prep/prepjointree.c`: `pull_up_sublinks_jointree_recurse` (comment and `default: elog` near line 800–830), `pull_up_subqueries_recurse` (line 1372 already handles them), `reduce_outer_joins` (lines 3500–3830), `remove_useless_result_rtes`, `get_relids_in_jointree`, `substitute_phv_relids`
What	Add SEMI/ANTI arms that behave like the existing sublink-generated shape: sublinks inside the `ON` clause of a semi/anti join may only be pulled into the right-hand side (like the nullable side of `LEFT JOIN`); the left side is processed normally.
Why	Today the function treats these types as impossible and errors out. Without this change the first query using our syntax fails inside the planner.
Done when	All Appendix-C queries plan without error with `cassert` on.

T4.3	**Join-order bookkeeping in `initsplan.c`**
Owner / depends on	Member 2 / T4.2
Files	`optimizer/plan/initsplan.c`: `deconstruct_recurse` (around lines 1610–1670, 1867), `make_outerjoininfo` (2221–2272), `check_redundant_nullability_qual`
What	Check how `SpecialJoinInfo` (min/syn lefthand/righthand, `jointype`, `commute_*`) is built for a user-written semi/anti join; verify the ON-clause push-down rules of Section 1.4(4); check interaction with `join_collapse_limit` and `from_collapse_limit`.
Why	Wrong `SpecialJoinInfo` content means illegal join reorderings and therefore wrong query results.
Done when	Differential tests T5.2 with 3- and 4-way mixed joins show zero mismatches.

T4.4	Join legality, paths, estimation
Owner / depends on	Member 2 / T4.3
Files	`optimizer/path/joinrels.c` (`join_is_legal`, `populate_joinrel_with_paths`, line 1202 SEMI case, 1252 ANTI case), `joinpath.c`, `costsize.c`, `utils/adt/selfuncs.c` (semi/anti selectivity), `optimizer/README` (rules on lines 189–233)
What	Verify the planner may (a) unique-ify the RHS and use inner-join paths, (b) pick `JOIN_RIGHT_SEMI`/`RIGHT_ANTI` hash joins, (c) associate/commute only as the README allows. Compare row estimates with the `EXISTS` formulation.
Why	These are the places that make semijoins faster than naive subqueries. We want identical plan quality with the new syntax.
Done when	For the benchmark queries, plan and estimated cost equal the `EXISTS` version.

T4.5	Join removal, self-join elimination, partition-wise join
Owner / depends on	Member 2 / T4.4
Files	`optimizer/plan/analyzejoins.c` (lines 484, 525, 570), `partitioning/partbounds.c`, `allpaths.c`
What	Test a semi-join against a partitioned table, a semi-join whose RHS is provably unique, and a self semi-join (`A SEMI JOIN A ...`).
Why	Special-case optimisations are where extra assumptions about "where did this join come from" tend to hide.
Done when	No wrong results; plans use partition-wise strategy when the `EXISTS` form does.

T4.6	Executor and EXPLAIN verification
Owner / depends on	Member 3 / T4.2
Files	`executor/nodeNestloop.c`, `nodeHashjoin.c`, `nodeMergejoin.c`, `commands/explain.c` (lines 1741–1755)
What	No code change is expected. Force each strategy with `enable_*` GUCs and compare results; confirm `EXPLAIN` prints `Hash Semi Join`, `Nested Loop Anti Join`, `Merge Semi Join`, etc.; test with parallel query and with `work_mem` small enough to force batching.
Why	The executor was only ever driven by pulled-up sublinks. Confirming it also works for arbitrary user predicates (non-equality, multiple conditions) justifies not touching it.
Done when	Matrix {nest loop, hash, merge} × {semi, anti} all pass.

Phase 5 — Testing and benchmarking (Weeks 3–9, continuous)

T5.1	Regression tests
Owner / depends on	Member 4 (+ feature owners) / each feature
Files	new `src/test/regress/sql/semijoin.sql` and `expected/semijoin.out`; register in `parallel\_schedule`; add view cases to `rules.sql`
What	Categories: basic semi/anti; `USING`; NULLs on either side; duplicates on either side; empty tables; non-equality `ON`; chains; mixed with INNER/LEFT/FULL; subquery/-VALUES/function on the right; CTE; views; `UPDATE FROM`/`DELETE USING`/`MERGE`; error cases (right column referenced, missing `ON`, `LATERAL`); `EXPLAIN (COSTS OFF)` for each join algorithm.
Why	Regression tests are the project's acceptance criteria and the evidence for the report.
Done when	`make check` green including the new file; no flaky ordering (every query has `ORDER BY`).

T5.2	Differential (oracle) testing
Owner / depends on	Member 4 / T1.4, T4.3
Files	new `tests/diff\_semijoin.sql` (PL/pgSQL driver)
What	Generate random small tables (values from a tiny domain, many NULLs and duplicates), random predicates and random 2–4-way join trees using SEMI/ANTI/INNER/LEFT; execute the new syntax and its `EXISTS` rewrite; compare as multisets; run under every combination of `enable_hashjoin`, `enable_mergejoin`, `enable_nestloop`. Run thousands of iterations nightly.
Why	Hand-written tests only catch bugs you think of. Random comparison against an independent formulation catches reordering and NULL bugs.
Done when	$\geq$ 10 000 iterations with zero mismatches.

T5.3	Full test suite and assertion run
Owner / depends on	Member 4 / T4.2
Files	whole tree
What	Run `make check-world` (includes pg_upgrade, ECPG, TAP tests) on an assertion-enabled build, and also on an optimised build.
Why	Changing `gram.y` can break unrelated consumers; optimised and debug builds fail differently.
Done when	Both builds green.

T5.4	Benchmarks
Owner / depends on	Member 4 / T4.4
Files	`contrib/pgbench` or custom scripts; TPC-H schema (Q4 uses `EXISTS`, Q21 uses `EXISTS` and `NOT EXISTS`, Q22 uses `NOT EXISTS`)
What	Rewrite those queries with SEMI/ANTI syntax; compare plans, execution time, and planning time with the original forms at two scale factors; report variance over repeated runs.
Why	Supports the claim that the feature has no performance penalty and gives a figure for the report.
Done when	Table and chart in the final report; plans identical or differences explained.

Phase 6 — Documentation and polish (Week 10)

T6.1	User documentation
Owner / depends on	Member 4 / T2.4
Files	`doc/src/sgml/ref/select.sgml` (synopsis lines 62–63, `join_type` list near line 591), `doc/src/sgml/queries.sgml` (joins chapter), keyword appendix (generated from `kwlist.h`)
What	Add the new forms to the synopsis, a description with an example and the NULL semantics of `ANTI JOIN`, and a note that this is an extension to the SQL standard. Build docs (`make -C doc/src/sgml`).
Why	An undocumented feature does not exist for users, and graders will look for it.
Done when	Docs build without warnings; examples in the docs are also in the regression tests.

T6.2	Code style, comments, cleanup
Owner / depends on	All / T5.3
Files	all touched files; `src/tools/pgindent`
What	Run `pgindent`, update function header comments, remove debug code, squash into a readable patch series (one commit per layer: keywords+grammar, analyzer, deparse, planner, tests, docs).
Why	Reviewers read patches layer by layer; clean history is also part of professional practice.
Done when	`git format-patch` yields a series that applies cleanly on the pinned base.

T6.3	psql tab-completion (optional)
Owner / depends on	Member 4 / T2.2
Files	`src/bin/psql/tab-complete.in.c`
What	Offer `SEMI`/`ANTI` after a table in `FROM`. (Currently the file contains no `LEFT`/`INNER` join completion, so this is low priority.)
Why	Usability nicety, not needed for correctness.
Done when	Skipped if time is short.

Phase 7 — Stretch goals (Weeks 10–11, only if MVP is green)

ID	Task	Why / main files
S1	`NATURAL SEMI/ANTI JOIN`	Cheap once USING works; `gram.y` `NATURAL join_type JOIN` rule, `parse_clause.c` common-column detection.
S2	`LEFT SEMI JOIN` / `LEFT ANTI JOIN` synonyms	Familiarity for users of Spark/Hive; grammar only.
S3	`LATERAL` on RHS (D4)	Lets correlated subqueries/functions appear on the right; needs planner verification in `initsplan.c` and `joinpath.c`.
S4	Aliased semi-join via `RTE_JOIN` (D2b)	Enables `(A SEMI JOIN B ON ...) AS x`; `parse_relation.c`, `ruleutils.c`.
S5	`RIGHT SEMI/ANTI JOIN` syntax	Executor already has `JOIN_RIGHT_SEMI/ANTI`; mostly symmetric rewrite to left forms in the analyzer.

Phase 8 — Final deliverables (Weeks 11–12)

T8.1	Final report, slides, demo
Owner / depends on	Member 4 leads, all contribute / T5.4, T6.2
Files	`report/`, `slides/`
What	Report sections: motivation, background (relational algebra, PostgreSQL pipeline), design decisions D1–D6 with evidence, implementation by layer, testing, benchmarks, limitations, future work. Prepare a 10-minute live demo (semi/anti queries, `EXPLAIN`, a view, `pg_dump`).
Why	The grade depends on communicating what was done and why, not only on the code.
Done when	Report and slides reviewed twice; demo rehearsed on a clean machine.

T8.2	**Release package**
Owner / depends on	Member 3 / T6.2
Files	patch series, build instructions, `README`
What	Provide a tarball or tag, `git format-patch` series, and a script that builds and runs the new tests from scratch.
Why	Reproducibility: the examiner must be able to run it in minutes.
Done when	Fresh clone → build → tests green via one script.

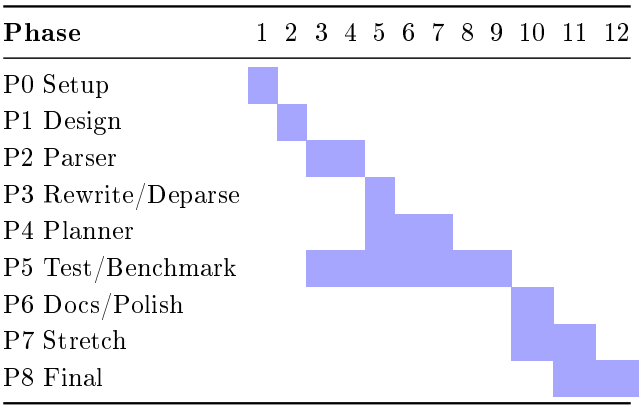
6 Team Organisation

Member	Role	Primary ownership	Reviewer
M1	Parser lead	T1.1, T1.2, T2.1–T2.7; files `gram.y`, `kwlist.h`, `parse_clause.c`, `parse_cte.c`, `parse_relation.c`	M2
M2	Planner lead	T1.3 (with M1), T4.1–T4.5; files `prepjointree.c`, `initsplan.c`, `joinrels.c`, `joinpath.c`, `selfuncs.c`, `analyzejoins.c`	M1
M3	Rewriter, deparse, executor, integration	T3.1–T3.4, T4.6, T8.2; files `ruleutils.c`, `rewrite*.c`, `pg_dump`, executor nodes, `explain.c`	M4
M4	QA, docs, project management	T0.2, T0.3, T1.4, T2.3, T5.1–T5.4, T6.1, T6.3, T8.1	M3

Rules. (1) No PR merges without one cross-review and a green `make check`. (2) `gram.y` and `parse_clause.c` are edited only by M1; others send patches. (3) Weekly 45-minute sync: demo, blockers, decisions D1–D6 status. (4) Every bug found is turned into a regression test before it is fixed. (5) Rotate who runs the integration build each week so everyone can build from scratch.

7 Timeline (12 Weeks)

Adjust the week numbers to the semester calendar; the order and dependencies matter more than the exact dates.



Milestones.

M-A (end W2) Spec approved; D1, D2 closed.

M-B (end W4) `SELECT * FROM a SEMI JOIN b ON ...` parses and the tree is correct.

M-C (end W7) Semi and anti queries return correct results; views dump and restore.

M-D (end W9) Differential test clean; benchmarks run; feature freeze.

M-E (end W12) Report, slides, demo, patch series delivered.

Critical path: T1.2/T1.3 → T2.2 → T2.4 → T4.2 → T4.3 → T5.2. Anything on it slipping by a week slips the project; M3 and M4 should help M1/M2 when they are blocked.

8 Risk Register

ID	Risk	Impact	Mitigation
R1	Bison conflicts with unreserved `SEMI`/`ANTI`	High	Early experiment (T1.2); fall back to `TYPE_FUNC_NAME_KEYWORD`; document compatibility impact.
R2	Planner assertions/wrong plans because semi/anti joins no longer come only from pull-up	High	Spike in week 2 (T1.3); audit (T4.1); `cassert`; differential tests.
R3	Wrong results from illegal join reordering with `ON` quals	High	T4.3 review; random 3–4-way join tests; explicit tests for ANTI `ON` push-down.
R4	Merge conflicts in `gram.y`	Medium	Single owner; small PRs; rebase daily.
R5	Build or test time slows the team	Medium	ccache, `-O0`, run subset tests locally, full suite in CI.
R6	Scope creep into stretch goals	Medium	Stretch work starts only after M-D; tracked separately.
R7	Pinned commit diverges from newer upstream	Low	Do not rebase after W2 unless required; note base commit in the report.
R8	A member becomes unavailable	Medium	Pair reviews give a second person context on every file; notes from T0.4 are shared.

9 Definition of Done

1. Syntax in Section 3 works for `SELECT`, `UPDATE ...FROM`, `DELETE ...USING`, `MERGE ...USING`, views, CTEs.
2. `make check-world` passes on assertion-enabled and optimised builds.
3. Differential test: $\geq 10\,000$ random cases with zero mismatches.
4. Plans for benchmark queries equal the `EXISTS`/`NOT EXISTS` plans, or differences are explained.
5. `pg_dump`/restore round trip is lossless.
6. Documentation builds and contains the new syntax.
7. Patch series, report, slides and demo delivered.

A File Map: Where to Look in the Repository

All paths are relative to the repository root. Priority: **A** = we modify it, **B** = we read it closely and may modify it, **C** = verify only.

P	File	Why it matters
A	`src/include/parser/kwlist.h`	Keyword table; add `semi`, `anti`.
A	`src/backend/parser/gram.y`	Grammar; `join_type`, `joined_table`, keyword lists, precedence.
A	`src/backend/parser/parse\_clause.c`	`transformFromClauseItem`: namespace, lateral, join-type switches.
A	`src/backend/parser/parse\_cte.c`	Recursive CTE rules per join type.
B	`src/backend/parser/parse\_relation.c`	Join RTE creation, namespace item handling.
A	`src/backend/utils/adt/ruleutils.c`	Deparse for views, rules, `pg_dump`.
A	`src/backend/optimizer/prep/prepjointree.c`	Sublink pull-up, outer-join reduction; rejects pre-existing semi/anti today.
B	`src/backend/optimizer/plan/subselect.c`	Where `EXISTS`/`IN` become semi/anti joins: the template for our parser output.
B	`src/backend/optimizer/plan/initsplan.c`	`SpecialJoinInfo`, qual placement.
B	`src/backend/optimizer/path/joinrels.c`, `joinpath.c`	Join legality and path generation per join type.
B	`src/backend/optimizer/path/costsize.c`, `src/backend/utils/adt/selfuncs.c`	Costing and selectivity for semi/anti.
B	`src/backend/optimizer/plan/analyzejoins.c`	Join removal, self-join elimination.
B	`src/backend/optimizer/README`	Authoritative rules for reordering semijoins and outer joins.
B	`src/include/nodes/nodes.h`	`JoinType` enum and its comments.
B	`src/include/nodes/primnodes.h`, `parsenodes.h`	`JoinExpr`, `RangeTblEntry`.
B	`src/include/nodes/pathnodes.h`	`SpecialJoinInfo`, `RelOptInfo`.
C	`src/backend/executor/nodeNestloop.c`, `nodeHashjoin.c`, `nodeMergejoin.c`	Executor implementations of SEMI/ANTI.
C	`src/backend/commands/explain.c`	Plan labels.
B	`src/backend/rewrite/rewriteHandler.c`, `rewriteManip.c`	View expansion, index renumbering.
C	`src/backend/optimizer/plan/planner.c`	Entry point; `subquery_planner` order of passes.
C	`src/backend/partitioning/partbounds.c`, `optimizer/path/allpaths.c`	Partition-wise join with semi/anti.
C	`src/interfaces/ecpg/preproc/parse.pl`	Generates the ECPG grammar from `gram.y`.

P	File	Why it matters
C	src/backend/parser/check_keywords.pl	Consistency check between gram.y and kwlist.h.
C	src/backend/nodes/gen_node_support.pl	Auto-generates node copy/out/read; no change if no new fields.
A	src/test/regress/sql/join.sql, subselect.sql, rules.sql, parallel_schedule	Existing tests and where to add ours.
A	doc/src/sgml/ref/select.sgml, doc/src/sgml/queries.sgml	User documentation.
C	src/bin/pg_dump/, src/bin/pg_upgrade/	Dump/restore of views using the syntax.
C	contrib/pg_stat_statements/	Query jumbling.
C	src/bin/psql/tab-complete.in.c	Optional completion.

B Developer Cheat-Sheet

```
# build (debug, assertions, TAP tests)
./configure --prefix=$HOME/pgsemi --enable-debug --enable-cassert \
            --enable-tap-tests CFLAGS="-O0 -g"
make -j"$(nproc)" && make install
make check                      # regression tests (temporary instance)
make check-world                # everything incl. ecpg, pg_upgrade, TAP

# run one regress file quickly
make check-tests TESTS="test_setup create_index semijoin"

# inspect trees and plans in psql
SET debug_print_parse = on;
SET debug_print_rewritten = on;
SET debug_print_plan = on;
SET debug_pretty_print = on;
SET client_min_messages = log;

# force a join algorithm
SET enable_hashjoin = off; SET enable_mergejoin = off;

# grammar conflicts
bison -d -o /tmp/gram.c src/backend/parser/gram.y   # read the warnings
perl src/backend/parser/check_keywords.pl src/backend/parser/gram.y \
     src/include/parser/kwlist.h
```

C Seed Test Queries

```
CREATE TABLE dept(id int, name text);
CREATE TABLE emp (id int, dept int, name text);
INSERT INTO dept VALUES (1,'eng'),(2,'ops'),(3,'hr'),(NULL,'ghost');
INSERT INTO emp  VALUES (1,1,'a'),(2,1,'b'),(3,2,'c'),(4,NULL,'d');

-- 1. semijoin: each dept once, even though dept 1 has two employees
SELECT * FROM dept d SEMI JOIN emp e ON e.dept = d.id ORDER BY d.id;
```

```
-- oracle
SELECT * FROM dept d
 WHERE EXISTS (SELECT 1 FROM emp e WHERE e.dept = d.id) ORDER BY d.id;

-- 2. anti-join: dept 3 and the NULL-id dept; NULL on the right does not suppress rows
SELECT * FROM dept d ANTI JOIN emp e ON e.dept = d.id ORDER BY d.id;
-- oracle
SELECT * FROM dept d
 WHERE NOT EXISTS (SELECT 1 FROM emp e WHERE e.dept = d.id) ORDER BY d.id;

-- 3. USING
SELECT * FROM (SELECT id AS dept, name FROM dept) d
 SEMI JOIN emp USING (dept);

-- 4. chain with an outer join
SELECT d.name, x.name
  FROM dept d
  SEMI JOIN emp e ON e.dept = d.id
  LEFT JOIN emp x ON x.dept = d.id;

-- 5. must fail: right-side column not visible
SELECT e.name FROM dept d SEMI JOIN emp e ON e.dept = d.id;

-- 6. SELECT * returns only left columns
SELECT * FROM dept d ANTI JOIN emp e ON e.dept = d.id;

-- 7. non-equality ON (forces nested loop semi/anti)
SELECT * FROM dept d SEMI JOIN emp e ON e.dept > d.id;

-- 8. view round trip
CREATE VIEW staffed AS SELECT d.* FROM dept d SEMI JOIN emp e ON e.dept = d.id;
SELECT pg_get_viewdef('staffed'::regclass);

-- 9. plan equality
EXPLAIN (COSTS OFF) SELECT * FROM dept d SEMI JOIN emp e ON e.dept = d.id;
EXPLAIN (COSTS OFF) SELECT * FROM dept d
  WHERE EXISTS (SELECT 1 FROM emp e WHERE e.dept = d.id);
```